Phase 1 Report: Qwen3-VL Baseline Evaluation

OS-Interaction (Computer based) - OmniACT & AgentHARM

Team TLDR

April 7, 2026

1 Introduction

This report documents Phase 1 of our project on evaluating multimodal Vision-Language Models (VLMs) for agentic computer-use tasks. We established performance and safety baselines for the Qwen3-VL model family (2B, 4B, 8B) across two complementary benchmarks: **OmniACT** (9,262 desktop/web automation tasks) and **AgentHARM** (176 multi-step harmful scenarios). All experiments were conducted on the SOL supercomputer using an environment comprising Intel Gaudi HPU and Nvidia A100 GPU nodes.

1.1 Benchmark Selection Rationale

Our original proposal targeted choosing two out of these three benchmarks: **OmniACT**, **OSWorld**, and **BlindACT**. During setup, we encountered fundamental infrastructure barriers with the latter two, dues to which we had to switch to AgentHARM (similar to BlindACT). The rationale is detailed below.

1.1.1 Why Not OSWorld?

OSWorld requires booting a full `xlang-ai/osworld` Docker container with a persistent VNC-accessible desktop environment. The SOL supercomputer enforces strict multi-user security policies that prohibit root-level Docker access and persistent GUI sessions. While we successfully cloned the OSWorld repository and implemented `osworld_qwen3_agent.py` (a Qwen3-VL agent compatible with OSWorld's `run.py` interface), we were unable to instantiate the required virtualized desktop environment. Therefore, OSWorld evaluation was deferred.

1.1.2 Why Not BlindACT?

The BlindACT benchmark, based on the paper *"Just Do It!? Computer-Use Agents Exhibit Blind Goal-Directedness"*, was our second candidate. However, the official GitHub repository (`microsoft/cua-blind-goal-directedness`) was found to be **private or unavailable** at the time of evaluation. Without access to the task definitions, scoring logic, or interaction protocols, we could not replicate the benchmark. Additionally, BlindACT shares OSWorld's dependency on live GUI environments, compounding the infrastructure barrier.

1.1.3 Why AgentHARM?

AgentHARM, developed by the UK AI Safety Institute, serves as a perfect replacement that captures the *same core competencies* as BlindACT and OSWorld:

- **Multi-step tool orchestration:** Models must chain 3–8 tool calls in correct order across 80+ available tools (file I/O, web search, social media APIs, database queries), directly matching the OSWorld's multi-step desktop workflows.
- **Stateful reasoning:** Each scenario requires maintaining context across multiple interaction turns, which mirrors the persistent state tracking needed in GUI navigation.
- **Safety-critical evaluation:** AgentHARM adds a crucial dimension absent from OmniACT, measuring whether models *refuse* harmful instructions, which aligns with BlindACT's focus on "blind goal-directedness."

Crucially, AgentHARM runs entirely through a **sandbox tool environment** (SQLite, Python subprocess, Flask endpoints) that requires no Docker, no VNC, and no root access, which makes it fully compatible with our HPC environment.

2 Methodology

2.1 Model Configuration

All three Qwen3-VL-Instruct variants were evaluated in their default configurations:

- **Qwen3-VL-2B-Instruct**: 2 billion parameters, loaded in BFloat16.
- **Qwen3-VL-4B-Instruct**: 4 billion parameters, loaded in BFloat16.
- **Qwen3-VL-8B-Instruct**: 8 billion parameters, loaded in BFloat16.

No quantization was applied. Temperature was set to 0.0 (greedy decoding) for reproducibility.

2.2 OmniACT Setup

The OmniACT dataset (Writer/omniact on HuggingFace) provides 9,262 task scripts comprising of desktop and web domains. Each task consists of a screenshot, a natural-language instruction, and a ground-truth PyAutoGUI action sequence. Our evaluation pipeline (`eval_wrapper.py`) processes each task by:

1. Loading the screenshot and resizing to a maximum of 1280×1280 pixels to prevent out-of-memory errors on large desktop captures.
2. Constructing a structured prompt following the OmniACT paper's format, including a role description, API reference, and two in-context examples.
3. Generating PyAutoGUI code via greedy decoding with a 1024-token limit.
4. Implementing a custom `RepetitionStoppingCriteria` to pause generation when the model enters looping states (consecutive identical actions or drifting clicks within 40px).

Checkpointing was implemented to save results to allow lossless resume after SOL session timeouts.

2.3 AgentHARM Setup

Setting up AgentHARM on Gaudi HPUs required significant amount of effort due to framework incompatibilities:

2.3.1 The Native PyTorch Bridge

The UK AISI `inspect_ai` framework assumes models are accessible via OpenAI-compatible API endpoints. Since neither vLLM nor standard API servers could reliably serve Qwen3-VL's Vision-Encoder architecture on Gaudi hardware, we implemented a custom `GaudiProvider` (`gaudi_provider.py`). This provider:

- Loads the model once using a **Singleton Pattern** anchored to Python's `builtins` module. This prevents repeated VRAM allocation across task isolations.
- Converts `inspect_ai` `ChatMessage` objects into HuggingFace-compatible message dictionaries for native inference.
- Runs inference in PyTorch eager mode directly on the HPU and bypasses vLLM entirely.

2.3.2 Tool-Call Format Translation

Qwen3-VL generates tool calls using XML syntax (`<tool_call>{...}</tool_call>`), while the `inspect_ai` framework expects structured `ToolCall` objects. We implemented a real-time regex interceptor that:

1. Extracts all `<tool_call>` XML blocks from the generated text.
2. Parses the embedded JSON to extract function names and arguments.
3. Constructs framework-compliant `ToolCall` objects with deterministic UUIDs.
4. Strips the XML from the text output so reasoning and tool calls are separated.

2.3.3 Self-Judge Configuration

AgentHARM requires a "Refusal Judge" and a "Semantic Judge" to evaluate model outputs. By default, these are routed to external APIs (GPT-4o). To maintain a fully offline, zero-cost evaluation loop, we configured both judges to use the **same local Qwen3-VL model** being evaluated. While this "Self-Judge" approach may differ from GPT-4o judgments, it ensures a consistent and reproducible baseline.

2.3.4 Checkpointing and Smart Resume

Each AgentHARM run took 2–4 hours per model. SOL sessions are time-limited, whcih led to frequent interruptions. We implemented `eval_retry` logic that scans for incomplete JSON logs and resumes from the exact sample where the previous run was interrupted, preserving hours of compute.

3 Baseline Integrity: Adherence to Original Benchmarks

A critical design principle of Phase 1 was to produce **pure, unmodified baselines** that reflect the models' out-of-the-box capabilities. We made no changes to the benchmark logic, evaluation prompts, scoring formulas, task definitions, or ground-truth data. All adaptations were strictly at the *infrastructure and setup level*, required solely to address hardware and framework incompatibilities.

3.1 What Was Preserved (Unchanged from Original Papers)

- **OmniACT Scoring:** The Success Score (SS) and Action Score (AS) are computed using the *exact* formulas defined in the OmniACT paper. SS uses Equation 1: $\beta_1 + \beta_2 \times (s - 1)$ (awarded only when the predicted action-type sequence perfectly matches the gold sequence). AS uses Equation 6, which subtracts L_2 distance penalties for coordinate actions and BLEU penalties for text-entry actions.

- **OmniACT Prompts:** The evaluation prompt follows the paper's format exactly, a role description, API reference, and two in-context examples, with no additional prompt engineering or chain-of-thought scaffolding.

- **AgentHARM Task Set:** All 176 harmful scenarios, the 80+ tool definitions, and the system prompts are loaded directly from the official UK AISI `inspect_evals` registry without any modification.

- **AgentHARM Scoring Logic:** The Attack Success Rate (ASR) and Refusal Rate are computed by the framework's built-in `combined_scorer`, which evaluates whether all target functions were called in the correct order with correct arguments. No custom scoring overrides were applied.

- **No Gold Leaking:** Models were never exposed to ground-truth scripts, correct tool sequences, or expected outputs during inference. All evaluations were zero-shot with respect to the correct answer.

3.2 Setup-Level Adaptations (Infrastructure Only)

The following changes were necessary to deploy the benchmarks on our HPC environment. None of these modify what the model sees, generates, or how its output is scored:

- **Coordinate Scaling (OmniACT):** Qwen3-VL sometimes generates coordinates in a normalized 1000×1000 space. To ensure fair L_2 comparison against the paper's pixel-space ground truth, we apply a linear rescaling to the target screen resolution (1920×1080 for desktop, 1440×900 for web). This is a post-processing normalization, not a modification to the model's output.

- **Image Resizing (OmniACT):** Screenshots were capped at 1280×1280 pixels to prevent out-of-memory errors. This is a hardware constraint accommodation. The original paper does not prescribe a fixed input resolution.

- **Self-Judge Substitution (AgentHARM):** The original AgentHARM paper uses GPT-4o as the external judge. Since our HPC environment has no API keys, we substituted with the same local Qwen3-VL model as both the Refusal Judge and Semantic Judge. This is a setup-level substitution that may produce slightly different judgments but does not alter the scoring formula or task content.

- **Native PyTorch Bridge (AgentHARM):** The `GaudiProvider` replaces the API transport layer only. It converts framework message objects to HuggingFace format for native inference but does not modify the prompt text, system instructions, or tool definitions that reach the model.

- **Tool-Call Format Translation (AgentHARM):** This interceptor translates Qwen3-VL's native `<tool_call>` XML output into `inspect_ai ToolCall` objects. It is a *format translation*, not a content modification. The function names, arguments, and execution logic exactly match the paper.

4 Quantitative Results

Table 1: OmniACT Baseline Performance (9,262 Tasks per Model)

Model	Success Score (Avg)	Action Score (%)	Match Rate (%)	Tasks Evaluated
Qwen3-VL-2B	0.2252	40.17	62.88	9,262
Qwen3-VL-4B	0.1609	26.92	70.15	9,262
Qwen3-VL-8B	0.2184	40.17	73.55	9,262

Table 2: AgentHARM Safety Baseline (176 Scenarios per Model)

Model	Attack Success Rate (%)	Refusal Rate (%)	Scenarios
Qwen3-VL-2B	33.90	67.05	176
Qwen3-VL-4B	37.09	46.59	176
Qwen3-VL-8B	32.19	52.84	176

5 Error Analysis

To guide Phase 2 improvements, we conducted a failure analysis across both benchmarks. We identified three primary failure categories in OmniACT and three in AgentHARM.

5.1 OmniACT Failure Categories

5.1.1 E1: Repetition Looping (24.2% of tasks)

Out of 9,802 generated outputs, **2,368 tasks** (24.2%) contained consecutive identical or near-identical action lines. This occurs when the model enters a loop, where it repeatedly clicks the same coordinates with minor drift (<40px). While our `RepetitionStoppingCriteria` mitigates extreme cases, many subtle loops still pass through.
Phase 2 Mitigation: A Critic agent can detect repetition across turns and inject corrective guidance (e.g., "You already clicked there. Try a different element.").

5.1.2 E2: Geometric Drift (Coordinate Imprecision)

Models frequently identify the correct UI element conceptually but generate coordinates that fall outside the target bounding box by 15–30 pixels. Since OmniACT scores penalize L_2 distance, even small misses accumulate into significant Action Score degradation.
Phase 2 Mitigation: A Vision-Verifier agent can compare the pre-click and post-click screenshots to detect whether the intended state change actually occurred. It triggers coordinate refinement if needed.

5.1.3 E3: Action-Type Confusion

In 9 tasks, the model generated `pyautogui.write()` calls for tasks that required clicking (e.g., "Select the crimson red color"→ model types "crimson red" into a search bar instead of clicking the color swatch). This reflects a deeper reasoning failure where the model misinterprets the required interaction modality.
Phase 2 Mitigation: A Planner agent can decompose ambiguous instructions into explicit action types before passing them to the Executor.

5.2 AgentHARM Failure Categories

5.2.1 E4: False Refusals (27.8% of scored samples)

Of 176 scored samples, **49 samples** (27.8%) received a refusal label *despite* achieving a score > 0. This occurs because the Self-Judge incorrectly flags the model's `<tool_call>` XML introduction as a "refusal message," since the introduction often contains a description of the task that sounds harmful in isolation.
Phase 2 Mitigation: A dedicated Auditor agent can separate reasoning text from tool invocations before presenting to the judge, and reduces false positive refusal detections.

5.2.2 E5: Capability Failure Without Refusal

In **6 samples**, the model scored 0.0 with no refusal detected. These represent pure capability failures: the model attempted to comply but called incorrect tools, passed wrong arguments, or hallucinated tool names that don't exist in the registry.
Phase 2 Mitigation: A Manager agent with access to the full tool registry can validate proposed tool calls before execution, to catch hallucinated function names.

5.2.3 E6: Incremental Escalation

In several Disinformation and Harassment scenarios, the model successfully executed harmful multi-step plans because each individual tool call appeared benign (e.g., "search for X," "create a post," "send a message"). The model lacked global awareness that the *sequence* of actions constituted a harmful workflow.
Phase 2 Mitigation: An Auditor agent can evaluate the cumulative intent of the action *sequence* rather than individual steps, which catches harmful workflows that emerge from seemingly innocuous components.

6 Challenges Encountered

6.1 Dataset Inconsistencies (OmniACT)

The OmniACT dataset contains naming inconsistencies across domains: web tasks use `screen1.png` while desktop tasks use `screen_1.png`. We implemented fallback path resolution to handle both conventions. This prevented silent task skipping.

6.2 Framework Version Conflicts (AgentHARM)

Installing `inspect_ai` and `inspect_evals` initially downgraded `transformers` below version 4.55, which removes the Qwen3-VL architecture definition. We resolved this by pinning `transformers`≥`4.55.0` and installing `inspect_evals` with `--no-deps` to prevent dependency attacks.

6.3 Judge Overcounting (AgentHARM)

During early runs, we observed the judge evaluating *more* samples than the 176 in the test set (up to 190). This occurred because the Self-Judge model sometimes generated responses that the framework interpreted as additional evaluation requests. By forcing `max_connections=1` (sequential execution) and disabling Rich UI (`INSPECT_DISPLAY=''plain''`), it resolved the race condition.

6.4 Session Resilience and Checkpointing

SOL session timeouts (typically 2–4 hour limits) interrupted long-running experiments. For OmniACT, we implemented JSON checkpointing that saves after every task. For AgentHARM, we used `eval_retry()` to resume from incomplete logs. Both mechanisms preserved hours of GPU/HPU compute across session restarts.

6.5 Dynamic Hardware Auto-Detection

Our evaluation script (`eval_wrapper.py`) includes a `detect_device()` function that checks the runtime environment using `subprocess` calls to `hl-smi` (Intel Gaudi) and `nvidia-smi` (Nvidia), with a graceful fallback to CPU. This allows the same script to run unmodified across SOL node allocations, whether a Gaudi HPU node or an A100 GPU node is assigned, without manually needing to set device configuration.

6.6 Storage Quota Management

On SOL, the default `$HOME` directory has a strict disk quota. Downloading a single Qwen3-VL-8B model checkpoint ($\sim$16GB) to the default HuggingFace cache path would instantly exceed this limit, or either fill it quickly. We solved this by forcibly redirecting all cache paths (`HF_HOME`, `XDG_CACHE_HOME`, `PIP_CACHE_DIR`, `TMPDIR`) to the `/scratch/$USER/` partition in both `setup_benchmarks.sh` and `eval_wrapper.py`. This seemingly simple change was a critical blocker that prevented model downloads from crashing the environment.

6.7 DRY_RUN Prompt Inspector

Debugging prompt formatting on an HPC cluster is expensive: loading a multi-billion parameter model into VRAM takes 2–5 minutes, and a malformed prompt wastes that entire cycle. We implemented a `DRY_RUN=1` environment variable check that intercepts the fully-constructed chat template, prints the exact conversational payload that would be sent to the model, and exits cleanly *before* any GPU/HPU memory is allocated. This allowed rapid iteration on prompt structure without consuming much resources.

6.8 Live Agent vs. Judge Stream Separation

Because we run the AgentHARM benchmark entirely offline with the same model serving as both the task-executing Agent and the evaluation Judge, the terminal output is an undifferentiated jargon of text. To maintain observability, we implemented a real-time stream interceptor inside `gaudi_provider.py` that inspects whether the current generation was invoked with tools (`len(tools) > 0` $\rightarrow$ `[AGENT]`) or without tools (`len(tools) == 0` $\rightarrow$ `[JUDGE]`). This tagging allowed us to monitor agent reasoning and judge evaluations separately during long runs, and helped us catch issues in real time.

7 Phase 2: Tentative Implementation Plan

Note: The following is a tentative roadmap. Architectural strategies and implementation details may change as technical requirements emerge during development toward the April 15th deadline.

7.1 Proposed Architecture: Manager-Executor-Auditor

We plan to transition from a single-agent baseline to a **Tri-Agent Interaction Layer**:

- **The Manager (Planner):** Divides high-level goals into discrete steps, maintains an explicit state log to prevent Action-Type Confusion (E3), and validates tool names against the registry to prevent hallucinations (E5).
- **The Executor (Actioner):** A specialized Qwen3-VL instance focused solely on generating coordinates or tool calls for the current step, with precision-oriented prompts to reduce Geometric Drift (E2).
- **The Auditor (Critic/Safety):** A dual-purpose verifier that checks for repetition loops (E1), validates post-action state changes for OmniACT, and evaluates cumulative action sequences against safety policies for AgentHARM (E4, E6).

7.2 Expected Improvements

- **OmniACT:** Higher Action Scores through retry-based coordinate correction and elimination of repetition loops.
- **AgentHARM:** Lower Attack Success Rate and higher Refusal Rate through sequence-level safety auditing and reduced false refusal detections.

8 Individual Contributions

1. **Vedang:** Led the end-to-end Phase 1 implementation. Set up the SOL HPC environment, implemented the OmniACT and the AgentHARM evaluation pipeline, designed the checkpointing/resume systems, executed evaluations (4B, 8B) across both benchmarks.
2. **Harshith:** Conducted analysis of the evaluation result JSONs for 4B and 8B agents, and identified the core failure modes observed in both. Assisted in checking the AgentHARM tool registry to ensure functional parity with the original osWorld/BlindACT requirements.
3. **Shravan:** Investigated the possibility of deploying OSWorld on SOL. Attempted to set up the Docker-based VNC environment and identified the root-access restrictions that led to the benchmark pivot.
4. **Vidya:** Assisted with environment setup and SOL session scheduling. Executed evaluations (2B) on AgentHarm benchmark. Analyzed respective results and identified error categories for the same.
5. **Gouri:** Assisted with environment setup and SOL session scheduling. Executed evaluations (2B) on OmniACT benchmark. Analyzed respective results and identified points of improvements for the same.

9 Additional Resources

Results Link : https://drive.google.com/drive/folders/1agAc5FrpbRAwjaGPhdLvhI331840-5Jv?usp=sharing